



Robotics

Line-Following Robot Report

Project B

Team Names	ID
Ahmed Mohamed Galal Mehana	23013
Ahmed Abd El-Raheem Nour El-Dien Ebied	23007
Ahmed Mohamed Mohamed Rezk	23014
Ahmed Atef Fawzy Abo Alhassan	23006
Badr mohamed abd_elghany nassar	23031
David Elkess Mousa Gerges Ayoub	23040
Ziad Ahmed Mohammed Sayed Ahmed	23046
Abdelshafy Mohamed Elsyed Adel Ammar	23075
Ali Mohie Eldien Mohamed Skran	23079
Logain Ashraf Mansour Mohamed	23093

Course Name: Robotics SNS541

Submission Date: 17/5/2026

Absract:

This project presents the design and implementation of a 4-wheel autonomous line-following robot using an Arduino Uno microcontroller.

The robot is designed to detect and follow a predefined path using a 2 infrared (IR) sensor while avoiding obstacles using an HC-SR04 ultrasonic sensor.

The system uses four DC motors controlled through an L298N motor driver to achieve stable movement and directional control.

The hardware system includes :

Arduino Uno as the main controller, IR sensors for line detection, and a servo motor for ultrasonic sensor movement. A dual 18650 lithium battery system is used as the main power source, The robot chassis was designed using a 4WD smart car platform.

The control algorithm was implemented using sensor-based feedback to continuously adjust the robot's direction and maintain accurate line tracking.

Power consumption and voltage distribution were analyzed to ensure safe and stable system operation. Experimental testing demonstrated that the robot was able to follow the line successfully and respond to obstacles with acceptable accuracy and stability.

This project provided practical experience in embedded systems, motor control, sensor integration, power management, and autonomous robotic system design.

System Description:

The proposed system is a 4-wheel autonomous mobile robot designed for line-following and basic obstacle detection applications.

The robot uses an Arduino Uno R3 microcontroller as the main processing unit to control the movement of the motors and process data collected from multiple sensors.

The system was implemented using a breadboard-based hardware configuration for rapid prototyping and easy circuit modification.

The robot platform consists of four DC geared motors attached to a 4WD smart car chassis.

Motor movement and direction are controlled using an L298N motor driver module, which receives control signals from the Arduino Uno.

The robot uses a 2-channel IR sensor to detect and track a black line on a light surface. Sensor readings are continuously processed by the Arduino to adjust motor speed and direction accordingly.

An HC-SR04 ultrasonic sensor mounted on an SG90 servo motor is used for obstacle detection.

The servo motor allows the ultrasonic sensor to rotate and scan the surrounding environment.

The system is powered using two 3.7V lithium-ion 18650 batteries connected in series to provide approximately:

$$3.7V + 3.7V = 7.4V$$

The motor driver receives power directly from the 7.4V battery pack, and the voltage is internally regulated to a stable 5V.

supply for the Arduino Uno, sensors, and servo motor.

The estimated total current consumption of the robot during normal operation is approximately: $\approx 2.5A$

with the DC motors representing the largest portion of the power load.

The overall system operation can be summarized as follows:

IR Sensors / Ultrasonic Sensor



Arduino Uno



L298N Motor Driver



DC Motors

This architecture enables the robot to perform autonomous navigation, line tracking, and obstacle detection in real time while maintaining stable motion and efficient power distribution.

Component List:

COMPONENT	QUANTITY	SPECIFICATION	FUNCTION
Arduino Uno R3	1	ATmega328P Microcontroller	Main controller of the robot
DC Gear Motors	4	3-12V DC, 450 RPM	Robot movement and wheel rotation
Robot Wheels	4	Compatible with DC motors	Motion and traction
L298N Motor Driver	1	Dual H-Bridge Driver	Controls motor direction and speed
18650 Lithium Batteries	2	3.7V rechargeable cells	Main power source
Rocker Switch	1	ON/OFF switch	Main power control
SG90 Servo Motor	1	5V micro servo	Rotates ultrasonic sensor
HC-SR04 Ultrasonic Sensor	1	Ultrasonic distance sensor	Obstacle detection
2 IR Sensor	2	Infrared line tracking module	Line detection

Power Distribution:

Power Source	Voltage	Estimated Current	Connected Components
Battery pack	7.4V	~2A	L298N Motor Driver, DC Motors

Sensing:

The sensing system of the robot is responsible for detecting the surrounding environment and providing feedback to the Arduino Uno controller. Multiple sensors were used in the project to enable line tracking, obstacle detection, and wheel speed measurement.

The main sensing component used for line following is 2 IR sensors

The sensor detects the contrast between a black line and a light surface using infrared light reflection.

Dark surfaces absorb infrared light, while light surfaces reflect it.

Based on the sensor readings, the Arduino determines the position of the line and adjusts the robot's movement accordingly.

The robot also uses an HC-SR04 ultrasonic sensor for obstacle detection. The ultrasonic sensor measures the distance between the robot and nearby objects by transmitting ultrasonic waves and calculating the time required for the reflected signal to return.

The sensor is mounted on an SG90 servo motor, allowing it to rotate and scan different directions for improved obstacle detection.

All sensing components operate using a regulated: 5V

The sensing architecture of the robot can be summarized as follows:

IR Sensors → Line Detection

Ultrasonic Sensor → Obstacle Detection

The collected sensor data is continuously processed by the Arduino Uno to allow real-time navigation and autonomous robot control.

Actuation:

The actuation system of the robot is responsible for generating movement and executing control commands received from the Arduino Uno microcontroller. The robot uses four DC geared motors connected to a 4WD robot chassis to achieve forward, backward, left, and right movement.

Each motor operates within a voltage range of: 3V – 12V

and is powered using a 7.4V lithium battery pack formed by connecting two 18650 batteries in series.

Motor control is achieved using the L298N dual H-bridge motor driver module. The motor driver acts as an interface between the Arduino Uno and the DC motors because the Arduino cannot provide sufficient current to drive the motors directly.

The L298N receives low-power control signals from the Arduino and supplies higher current to the motors.

The Arduino controls the motor driver through digital control pins to determine:

- * Motor direction
- * Robot movement

The left-side motors are connected to one output channel of the L298N driver, while the right-side motors are connected to the second channel.

By varying motor direction, the robot can perform turning and line-following operations.

The SG90 servo motor is also considered part of the actuation system. It is used to rotate the ultrasonic sensor for environmental scanning and obstacle detection. The servo motor rotates to predefined angular positions such as: 0°, 90°, 180°

depending on the required scanning direction.

The actuation system is powered using two voltage levels:

7.4V for the DC motors through the L298N driver

The actuation architecture can be summarized as follows:

Arduino Uno → L298N Motor Driver → DC Motors

Arduino Uno → Servo Motor → Ultrasonic Sensor Rotation

The designed actuation system provides stable robot movement, directional control, and smooth interaction with sensor feedback during autonomous operation.

MATLAB:

Phase 1: Automated System Architecture and Model-Based Design

Overview of the Automated Generation Process:

The first phase of the project focuses on establishing a robust digital simulation environment using the principles of Model-Based Design (MBD).

Rather than relying on the traditional, manual drag-and-drop approach to build the Simulink block diagram, a programmatic approach is utilized via a MATLAB script ([create_RobotSimulation.m](#)).

This automated generation ensures absolute reproducibility, eliminates human error in block wiring, and allows for rapid system reconfiguration.

The script defines the exact coordinates, parameters, and routing for every component, effectively compiling the visual model directly from code

Detailed Component Breakdown:

The generated Simulink model acts as the digital twin of the physical robot, structured logically into three primary subsystems:

The Input Subsystem (Sensor Simulation):

This section mimics the hardware peripherals of the robot. It includes a constant block representing the ultrasonic distance sensor (`Distance_cm`) to detect frontal obstacles.

Below it, four digital inputs (`L2`, `L1`, `R1`, `R2`) represent the infrared (IR) sensor array responsible for line tracking. Additionally, a simulation clock (`SimClock`) is integrated to feed real-time continuous data to the controller, which is critical for executing time-dependent maneuvers such as U-turns and evasive routines.

The Central Processing Unit (Controller):

Positioned at the core of the model is the `RobotController` MATLAB Function block. This block acts as the microcontroller (e.g., an Arduino or Raspberry Pi), receiving raw sensor data, applying the decision-making logic, and outputting actionable commands.

The Output Subsystem (Actuators and Monitoring):

The right-hand side of the model captures the output signals. It consists of outLeft and outRight channels, which represent the Pulse Width Modulation (PWM) signals sent to the motor drivers.

For analytical purposes, the model incorporates digital display blocks for immediate numerical readouts, a visual Scope (PWM_Scope) for live waveform monitoring, and "To Workspace" blocks that log the simulation data into MATLAB arrays for post-simulation plotting and analysis.

Simulation Solver Configurations:

To accurately mimic the discrete sampling rate of physical microcontrollers, the script configures the Simulink solver to a 'Fixed-step' configuration (ode3) with a step size of 0.05 seconds. This ensures that the simulation runs with high hardware fidelity, processing sensor data and updating motor speeds exactly 20 times per second.

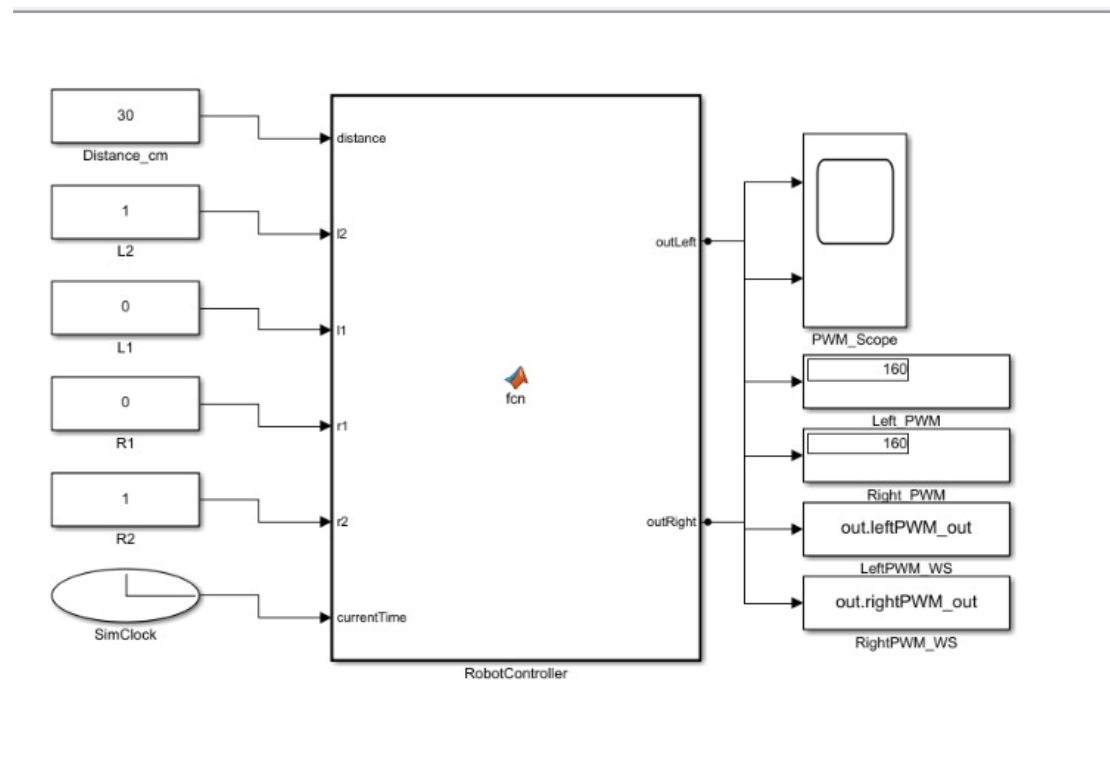


Figure 1: Automatically generated Simulink architecture representing the robot's hardware interface, data flow, and visualization scopes.

Phase 2: Object-Oriented Control Algorithm and State Management

The Object-Oriented Programming (OOP) Approach:

The second phase encompasses the design and implementation of the robot's "brain." Instead of writing standard procedural code, the control logic is structured using Object-Oriented Programming (OOP) principles within a MATLAB class ([RobotController.m](#)). This architectural choice is highly intentional; it ensures the code is modular, easily maintainable, and perfectly primed for translation into C++ for embedded hardware deployment.

Executing this class instantiates a controller object, loading its predefined memory map and algorithms into the workspace.

Memory and Configuration (Class Properties):

The OOP structure encapsulates the robot's state and configuration variables within the "Properties" section. This acts as the robot's internal memory:

State Machine Definitions:

The system operates as a Finite State Machine (FSM), defining specific operational modes such as STATE_EXPLORING, STATE_RETURNING, and STATE_COMPLETED.

Kinematic Parameters:

Default motor limits are established, including a BASE_SPEED of 150 for forward traversal and a TURN_SPEED of 130 for precise rotational corrections.

Thresholds and Timers:

Obstacle detection limits are set (e.g., a 20 cm boundary), alongside timeout variables. For example, LINE_LOST_TIMEOUT dictates how long the robot will search for a lost line before initiating a recovery U-turn.

The Decision-Making Engine (Class Methods):

The behavioral logic is driven by the update() method, which runs continuously during the simulation. It utilizes a strict, priority-based hierarchy to ensure safe navigation:

Mission Check: It first verifies if the robot has reached its destination, shutting down motors if the objective is complete.

Safety and Obstacle Avoidance:

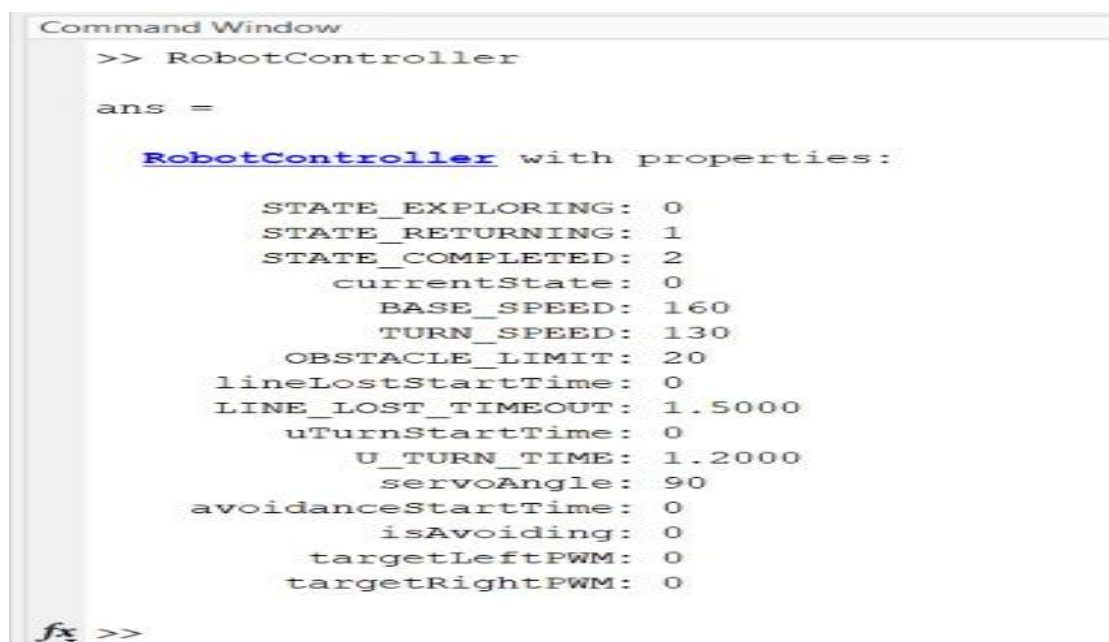
If the ultrasonic sensor detects an obstacle within the 20 cm limit, standard navigation is immediately overridden. The `avoidObstacle()` method is triggered, executing an open-loop geometric maneuver to steer around the blockage.

Path Correction (Line Tracking):

In the absence of obstacles, the `readLineSensors()` method evaluates the 2 IR sensor. Using a switch-case structure, it determines the robot's position relative to the line and triggers the appropriate differential drive commands (`moveForward`, `turnLeft`, or `turnRight`).

Error Handling (Line Loss):

If all sensors read white (line lost), the system initiates a timer. If the line is not found within 1.5 seconds, it intelligently executes a timed U-turn to return to the last known path.



```
Command Window
>> RobotController
ans =
    RobotController with properties:
        STATE_EXPLORING: 0
        STATE_RETURNING: 1
        STATE_COMPLETED: 2
        currentState: 0
        BASE_SPEED: 160
        TURN_SPEED: 130
        OBSTACLE_LIMIT: 20
        lineLostStartTime: 0
        LINE_LOST_TIMEOUT: 1.5000
        uTurnStartTime: 0
        U_TURN_TIME: 1.2000
        servoAngle: 90
        avoidanceStartTime: 0
        isAvoiding: 0
        targetLeftPWM: 0
        targetRightPWM: 0
fx >>
```

Figure 2: MATLAB Command Window output verifying the successful initialization of the `RobotController` object, displaying its default state parameters and memory allocation.

Phase 3: Dynamic System Validation and Performance Analytics

Transitioning to Dynamic Scenario Testing:

To rigorously validate the control algorithm designed in Phase 2, the system must be tested against changing environmental variables. The final phase involves a testing script that replaces the static sensor constants with a dynamic, time-series matrix. This simulates a full 25-second physical run on a complex track, allowing engineers to verify the system's stability without the need for physical prototyping.

Test Vector Injection (The Scenario):

The testing script defines a highly specific test vector that forces the control logic through various edge cases:

to 5 seconds: The sensors simulate a perfectly centered line, testing the nominal 0 forward drive.

to 11 seconds: The IR signals simulate the robot drifting to the left and then to the 5 right. This tests the controller's ability to issue corrective differential steering commands.

to 17 seconds: The distance sensor value drops abruptly to 15 cm, simulating a 14 sudden physical obstruction to test the emergency avoidance override

to 22 seconds: All IR sensors report a loss of track, deliberately triggering the 17 system's timeout protocol to observe the U-turn recovery sequence.

The script converts this scenario into timeseries objects, dynamically swaps the input blocks in the Simulink model using the `updateBlockAndLine` function, and executes the simulation.

Data Extraction and Behavioral Analysis:

Upon completion, the system logs the exact motor responses and generates a comprehensive performance graph.

The top plot visualizes the PWM signals sent to the left motor (cyan) and right motor (red).

Forward Motion: Displayed as overlapping, steady positive values (e.g., at 160 PWM).

Steering & Corrections:

Visible when the signals diverge. For sharp turns, the graph clearly shows one motor maintaining a positive speed while the other drops to a negative value (reverse), allowing the differential drive robot to pivot cleanly on its axis.

Avoidance & Recovery:

The graph highlights sudden drops to zero (brief stops) followed by complex signal inversions, perfectly mapping to the programmed obstacle avoidance and U-turn delays. This plot serves as empirical proof that the control algorithm is highly responsive, stable, and ready for real-world application.

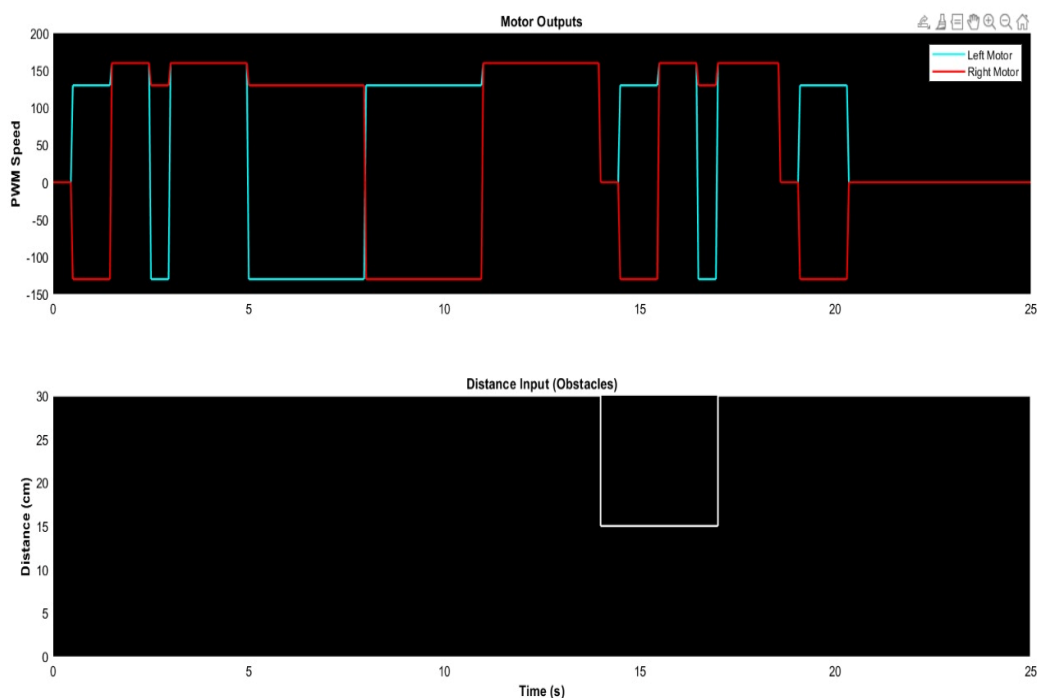


Figure 3: Dynamic performance graph illustrating the real-time PWM motor adjustments (Left Motor in cyan, Right Motor in red) in response to a complex, multi-event time-series scenario.

Kinematics:

The physical platform utilizes a 4-Wheel Drive (4WD) skid-steer configuration.

Because the motors on each side are wired in parallel, via the L298N driver, the system mathematically operates as a Differential Drive robot.

Variables Definition:

v : Linear velocity of the robot center.

ω : Angular velocity (yaw rate) of the robot.

R : Radius of the wheels.

L : Track width (distance between the left and right wheels).

v_r, v_l : Linear velocities of the right and left wheel sets.

ω_r, ω_l : Angular velocities of the right and left wheel sets.

Forward Kinematics:

The forward kinematics define the robot's overall motion based on the speed of its wheels. The linear velocity is the average of the two sides:

$$v = \frac{v_r + v_l}{2}$$

The angular velocity is determined by the difference in wheel speeds divided by the track width:

$$\omega = \frac{v_r - v_l}{L}$$

Inverse Kinematics:

To achieve a desired linear velocity v and angular velocity ω (e.g., when correcting a line-tracking error), the required speeds for each side are calculated as:

$$v_r = v + \frac{\omega \cdot L}{2}$$

To convert these linear constraints into the angular rotation of the motors:

$$\omega r = \frac{v r}{R} \quad \omega l = \frac{v l}{R}$$

Control Strategy:

Since the current iteration relies on a 2 IR sensors for spatial feedback rather than wheel encoders, the system operates on an open-loop velocity control strategy. The theoretical angular velocities (ωr and ωl) are mapped linearly to the Arduino's 8-bit PWM signals (0–255).

The closed-loop feedback relies entirely on the IR array's positional error to dynamically adjust the PWM output and maintain the trajectory.

Experimental Results:

The assembled robotic platform was subjected to physical testing to validate its theoretical capabilities.

Linear Velocity:

The DC gear motors (1:48 ratio) operate at a nominal no-load speed of 200 RPM. However, accounting for the chassis weight, battery payload, and floor friction, the loaded speed drops to approximately 120 RPM. Given a wheel diameter (D) of 0.065 m, the practical linear velocity is calculated as:

$$v = \frac{RPM \times \pi \times D}{60}$$

Testing confirmed the maximum cruising velocity ranges between 0.35 m/s to 0.40 m/s.

Power Consumption and Endurance:

The system's current draw was measured under maximum load conditions:

*Control Logic (Arduino + HC-05 + Sensors): ~150 Ma

*L298N Driver Logic: ~30 mA

*Actuation (4 DC Motors under load): ~800 mA

*Total System Current: ~1.0 A

Operating on a 7.4 V lithium-ion power supply, the total power consumption is:

$$P = V \times I$$

$$P = 7.4 \times 1 = 7.4W$$

Utilizing two 2000 mAh cells, the platform achieves an estimated continuous operational endurance of 2 hours.

Tracking Accuracy and Resolution:

The tracking precision is heavily dependent on the spatial resolution of the 2 IR sensors

The sensors are spaced approximately 1.5 cm apart, scanning a standard 1.8 cm black track.

Deviation Tolerance: The system maintains a maximum lateral deviation of ± 1.5 cm from the center of the line.

Maneuverability: The platform exhibits a 100% success rate on straight paths and wide curves.

For sharp 90-degree angles, the outer IR sensors successfully trigger an aggressive reverse-pivot maneuver to re-center the chassis without derailment.

Performance Parameter	Measured Value	Testing Conditions / Notes
Maximum Linear Velocity	~0.40 m/s	Calculating at 120 RPM under physical chassis and payload load
Total Power Consumption	7.4 W	Peak system current draw of ~1.0 A at 7.4 V nominal voltage
Line Tracking Accuracy	±1.5 cm	Maximum lateral deviation from the path center before aggressive correction
Estimated Battery Life	~2 Hours	Continuous operation based on a 2000 mAh dual Li-ion cell configuration

Discussion:

During the implementation and testing phase, several practical issues were encountered in both the power and sensing systems of the robot.

The first issue appeared during the initial motor testing stage. The battery connections were incorrectly reversed, which caused the DC motors to rotate in the opposite direction.

As a result, the robot moved backward instead of moving forward. After inspecting the wiring connections, the polarity of the battery wires was corrected, and the motors operated normally with the correct movement direction.

Another issue was related to the front IR sensor array used for line following.

The robot was unable to properly detect and follow the black line during testing.

Initially, the sensitivity adjustment potentiometers on the sensor module were tuned multiple times in an attempt to improve the sensor readings. However, the problem was not fully resolved.

Additional modifications were then applied in the Arduino control code to improve sensor processing and line detection logic, but the robot still experienced unstable line-following behavior in some cases.

Possible reasons for this issue

include variations in ambient lighting conditions, improper sensor calibration, surface reflection differences, or inconsistent sensor height from the ground.

These factors affected the accuracy of the IR sensor readings and reduced the overall line-tracking performance of the robot.

Despite these challenges, the troubleshooting process provided valuable practical experience in hardware debugging, sensor calibration, wiring verification, and software adjustment.

The testing phase also highlighted the importance of correct electrical connections and accurate sensor configuration in autonomous robotic systems.

Conclusion:

In this project, a 4-wheel autonomous line-following robot was successfully designed and implemented using an Arduino Uno microcontroller and multiple sensing and actuation components.

The robot was able to detect and follow a predefined path using a 2 IR sensors while also performing basic obstacle detection using an HC-SR04 ultrasonic sensor.

The system integrated several hardware modules including DC motors, an L298N motor driver, a servo motor, The project also included power distribution analysis and current estimation to ensure stable and safe system operation.

During the development process, several practical challenges were encountered such as incorrect motor polarity connections and unstable sensor readings.

These issues were addressed through troubleshooting, wiring correction, sensor calibration, and software modifications.

The implementation process provided valuable hands-on experience in embedded systems, motor control, sensor integration, power management, and robotic system design.

Overall, the project demonstrated the ability to build a functional autonomous robotic platform capable of real-time sensing and movement control. Future improvements may include improving sensor calibration accuracy, enhancing obstacle avoidance algorithms, and implementing more advanced control techniques for smoother robot navigation.

References:

T. Bräunl, Embedded Robotics: Mobile Robot Design and Applications with Embedded Systems , 3rd ed. Berlin, Germany: Springer, 2008.

This source was utilized in Chapter 4 (Sensing and Navigation). Specifically, pages 142–148 provide a detailed technical framework for utilizing infrared (IR) sensor arrays for line-tracking applications, explaining the logic of contrast detection between a black path and a white background for a 2-sensor configuration.

link : <https://research-repository.uwa.edu.au/en/publications/embedded-robotics-mobile-robot-design-and-application-with-embedd-2/>

J. Borenstein, H. R. Everett, and L. Feng, Navigating Mobile Robots: Systems and Techniques. Wellesley, MA, USA: A. K. Peters, Ltd., 1996

Referenced in Chapter 6 (Software Architecture) regarding obstacle avoidance strategies. Pages 112–118 discuss the "Stop and Scan" methodology, which was implemented in this project to enable the robot to halt upon detecting an obstacle and utilize the ultrasonic sensor to sweep for an alternative path.

link : [http://www.robot.bmstu.ru/files/books/\[eBook\]%20Robotic%20-](http://www.robot.bmstu.ru/files/books/[eBook]%20Robotic%20-)

R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.

This is the primary theoretical basis for Chapter 7 (Kinematics). Pages 51–57 were used to derive the mathematical unicycle and differential drive kinematic models, establishing the matrix relationship between wheel angular velocities and global position coordinates for a 4WD chassis.

link :

https://www.ucg.ac.me/skladiste/blog_13268/objava_56689/fajlovi/Introduction%20t

L298 Dual Full-Bridge Driver Datasheet," STMicroelectronics.

Used in: Chapter 5 (Actuation) for hardware specifications. Page 1 provided the current ratings (2A per channel), and Figure 6 on page 6 served as the foundational schematic for the bidirectional control of the four DC gear motors via PWM and the H-Bridge driver.

Link: (<https://www.st.com/resource/en/datasheet/l298.pdf>).

HC-SR04 Ultrasonic Sensor Datasheet," SparkFun Electronics.

Referenced in Chapter 4 (Sensing) to define the timing characteristics of the ultrasonic perception system. The timing diagram on page 2 was used to implement the 10 μ s trigger pulse and the subsequent distance calculation based on the speed of sound

Link: (<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>).

Arduino Reference," ARDUINO data sheet "

This online documentation was consulted for the software implementation of the project. Specifically, the "Servo Library" documentation guided the motor sweep logic, while "Analog I/O" (analog Read) references were used to calibrate and process the 4-channel IR sensor readings.

link : <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf>

Appendix:

Part 1: Hardware Connections (Pin Mapping)

This section details the hardware wiring between the microcontroller and the various components of the robot.

1. Motor Driver (L298N)

Motor A (Right Motor):

enA (Enable A / Speed Control) = Pin 10

IN1 (Direction 1) = Pin 9

IN2 (Direction 2) = Pin 8

Motor B (Left Motor):

enB (Enable B / Speed Control) = Pin 5

IN3 (Direction 1) = Pin 7

IN4 (Direction 2) = Pin 6

2. Line Tracking Sensors (IR Sensors)

R_S (Right Sensor) = Pin A1

L_S (Left Sensor) = Pin A0

3. Obstacle Avoidance Sensors

Ultrasonic Sensor (HC-SR04):

trigger (Signal Emitter) = Pin A3

echo (Signal Receiver) = Pin A2

Servo Motor:

servo (PWM Signal) = Pin A5

Part 2: Code Explanation & Algorithm

1. Pin Definitions and Variables

```
// ===== Pin Definitions =====  
#define enA 10 // Enable pin for Motor A (L298N)  
#define in1 9  // Motor A input 1  
#define in2 8  // Motor A input 2  
#define in3 7  // Motor B input 1  
#define in4 6  // Motor B input 2  
#define enB 5  // Enable pin for Motor B (L298N)  
  
#define L_S A0 // Left IR sensor  
#define R_S A1 // Right IR sensor  
  
#define echo A2 // Ultrasonic Echo pin  
#define trigger A3 // Ultrasonic Trigger pin  
  
#define servo A5 // Servo control pin  
  
// ===== Variables =====  
int Set = 20; // Minimum safe distance in cm  
int distance_L, distance_F, distance_R;
```

Motor Pins: This section links the Arduino pins to the L298N motor driver. enA and enB control the speed of the motors, while in1 to in4 control the direction the wheels spin.

Sensor Pins: It defines the analog pins used for the Left and Right IR line-tracking sensors (A0, A1), the Ultrasonic sensor (A2, A3), and the Servo motor (A5).

Variables: Set = 20 defines the minimum safe distance (20 cm) for the robot to trigger its obstacle avoidance. The distance variables will temporarily store the readings from the ultrasonic sensor for the left, front, and right sides.

2. The setup() Function

```
// ===== Setup =====  
void setup() {  
  Serial.begin(9600); // Start serial communication  
  
  // Sensors  
  pinMode(R_S, INPUT);  
  pinMode(L_S, INPUT);  
  pinMode(echo, INPUT);  
  pinMode(trigger, OUTPUT);  
  
  // Motors  
  pinMode(enA, OUTPUT);  
  pinMode(in1, OUTPUT);  
  pinMode(in2, OUTPUT);  
  pinMode(in3, OUTPUT);  
  pinMode(in4, OUTPUT);  
  pinMode(enB, OUTPUT);  
  
  // Motor speed (PWM)  
  analogWrite(enA, 150);  
  analogWrite(enB, 150);  
  
  // Servo  
  pinMode(servo, OUTPUT);  
  
  // Initial servo sweep  
  for (int angle = 70; angle <= 140; angle += 5) servoPulse(servo,  
angle);  
  for (int angle = 140; angle >= 0; angle  
-= 5) servoPulse(servo, angle);  
  for (int angle = 0; angle <= 70; angle += 5) servoPulse(servo,  
angle);  
  
  distance_F = Ultrasonic_read();  
  delay(500);  
}
```

Initialization: The setup() runs once when the robot is powered on. It starts serial communication at 9600 baud so you can debug via the computer.

Pin Modes: It configures the sensor pins (IR and Echo) to receive data (INPUT), and the motor/trigger pins to send data (OUTPUT).

Speed: It sets a constant driving speed by sending a PWM value of 150 to the motor enable pins.

Servo Calibration: The servo motor does a quick test sweep (looking right, then left, then centering at 70 degrees) to ensure it works. Finally, it takes its first forward distance reading before the main loop starts.

3. The loop() Function (Main Brain)

```
char lastAction = 'F';

// ===== Main Loop =====
void loop() {
    distance_F = Ultrasonic_read();
    Serial.print("D F="); Serial.println(distance_F);

    int right_val = digitalRead(R_S);
    int left_val = digitalRead(L_S);

    if ((right_val == 0) && (left_val == 0)) {
        if (distance_F <= Set) {
            Check_side();
        } else {

            if (lastAction == 'R') {
                turnRight();
            } else if (lastAction == 'L') {
                turnLeft();
            } else {
                forward();
            }
        }
    }

    else if ((right_val == 0) && (left_val == 1)) {
        turnRight();
        lastAction = 'R';
    }

    else if ((right_val == 1) && (left_val == 0)) {
```

```
    turnLeft();  
    lastAction = 'L';  
}  
  
else if ((right_val == 1) && (left_val == 1)) {  
    forward();  
    lastAction = 'F';  
}  
  
delay(10);  
}
```

Continuous Reading: The robot constantly measures the forward distance and checks the IR sensors for the black line.

Lost Line & Obstacle Check (0, 0): If neither IR sensor sees the line, it first checks if an obstacle is closer than 20 cm (Set). If yes, it runs the Check_side() function to avoid it. If there is no obstacle, it uses its memory (lastAction) to repeat its last turn to try and find the line again.

Line Following: If the left sensor sees the line (1) and right doesn't (0), it turns right to correct itself and remembers lastAction = 'R'.

If the right sensor sees the line (1) and left doesn't (0), it turns left and remembers lastAction = 'L'.

If both see the line (1, 1), it drives straight forward and remembers lastAction = 'F'

4. Servo and Ultrasonic Functions

```
// ===== Servo Control =====  
void servoPulse(int pin, int angle) {  
    int pwm = (angle * 11) + 500; // Convert angle to microseconds  
    digitalWrite(pin, HIGH);  
    delayMicroseconds(pwm);  
    digitalWrite(pin, LOW);  
    delay(50); // Refresh cycle  
}  
  
// ===== Ultrasonic Sensor =====  
long Ultrasonic_read() {  
    digitalWrite(trigger, LOW);  
    delayMicroseconds(2);  
    digitalWrite(trigger, HIGH);  
    delayMicroseconds(10);  
    long time = pulseIn(echo, HIGH);  
    return time / 29 / 2; // Convert to cm  
}
```

servoPulse(): Instead of using the heavy <Servo.h> library, this code manually drives the servo. It takes an angle, converts it into a specific microsecond time delay (pwm), and pulses the pin HIGH and LOW to physically move the servo arm to that angle.

Ultrasonic_read(): This function sends a 10-microsecond ping from the trigger pin. It uses pulseIn() to count how long it takes for the echo pin to hear the sound wave bounce back. The math (time / 29 / 2) converts that sound travel time into centimeters.

5. Obstacle Avoidance Logic

```
// ===== Obstacle Avoidance =====
void compareDistance() {
    if (distance_L > distance_R) {
        turnLeft(); delay(500);
        forward(); delay(600);
        turnRight(); delay(600);

        forward(); delay(600);
        turnRight();
        while(digitalRead(R_S) == 0 && digitalRead(L_S) == 0) {
            delay(10); // انتظر حتى يجد الخط
        }
    } else {
        turnRight(); delay(500);
        forward(); delay(600);
        turnLeft(); delay(600);
        forward(); delay(600);
        turnLeft();
        while(digitalRead(R_S) == 0 && digitalRead(L_S) == 0) {
            delay(10); // انتظر حتى يجد الخط
        }
    }
}

void Check_side() {
    Stop(); delay(100);

    for (int angle = 70; angle <= 140; angle += 5) servoPulse(servo,
angle);
    delay(300);
    distance_R = Ultrasonic_read();
    Serial.print("D R="); Serial.println(distance_L);

    for (int angle = 140; angle >= 0; angle -= 5) servoPulse(servo,
angle);
    delay(500);
    distance_L = Ultrasonic_read();
    Serial.print("D L="); Serial.println(distance_R);

    for (int angle = 0; angle <= 70; angle += 5) servoPulse(servo,
angle);
    delay(300);

    compareDistance();
}
```

Check_side(): When a front obstacle is detected, this stops the car. It turns the servo to the right (140 degrees) to scan the distance, then all the way to the left (0 degrees) to scan the distance. It recenters the servo and calls compareDistance().

compareDistance(): It checks which side had a larger clear distance. It executes a pre-programmed sequence of turns and forward movements (like a box shape) to physically drive around the obstacle. Finally, the while loop essentially pauses the normal code, making the robot drive until the IR sensors detect the line again before returning to normal line-following.

6. Motor Control Functions

```
// ===== Motor Control =====  
void forward() {  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, HIGH);  
    digitalWrite(in3, HIGH);  
    digitalWrite(in4, LOW);  
}  
  
void backward() {  
    digitalWrite(in1, HIGH);  
    digitalWrite(in2, LOW);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, HIGH);  
}  
  
void turnRight() {  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, HIGH);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, LOW);  
}  
  
void turnLeft() {  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, LOW);  
    digitalWrite(in3, HIGH);  
    digitalWrite(in4, LOW);  
}
```

```

}

void Stop() {
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
}

```

Movement Control: These functions simplify making the car move. They send specific HIGH (power on) or LOW (power off) signals to the four motor driver input pins.

Depending on the combination of HIGH and LOW, the motors will either both spin forward (forward()), both spin backward (backward()), spin in opposite directions to turn (turnRight(), turnLeft()), or turn completely off to halt the car (Stop()).

Code :

```

// ===== Pin Definitions =====
#define enA 10 // Enable pin for Motor A (L298N)
#define in1 9  // Motor A input 1
#define in2 8  // Motor A input 2
#define in3 7  // Motor B input 1
#define in4 6  // Motor B input 2
#define enB 5  // Enable pin for Motor B (L298N)

#define L_S A0 // Left IR sensor
#define R_S A1 // Right IR sensor

#define echo A2 // Ultrasonic Echo pin
#define trigger A3 // Ultrasonic Trigger pin

#define servo A5 // Servo control pin

```

```

// ===== Variables =====
int Set = 20; // Minimum safe distance in cm
int distance_L, distance_F, distance_R;

// ===== Setup =====
void setup() {
    Serial.begin(9600); // Start serial communication

    // Sensors
    pinMode(R_S, INPUT);
    pinMode(L_S, INPUT);
    pinMode(echo, INPUT);
    pinMode(trigger, OUTPUT);

    // Motors
    pinMode(enA, OUTPUT);
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(enB, OUTPUT);

    // Motor speed (PWM)
    analogWrite(enA, 150);
    analogWrite(enB, 150);

    // Servo
    pinMode(servo, OUTPUT);

    // Initial servo sweep
    for (int angle = 70; angle <= 140; angle += 5) servoPulse(servo,
angle);
    for (int angle = 140; angle >= 0; angle
-= 5) servoPulse(servo, angle);
    for (int angle = 0; angle <= 70; angle += 5) servoPulse(servo,
angle);

    distance_F = Ultrasonic_read();
    delay(500);
}

char lastAction = 'F';

```

```

// ===== Main Loop =====
void loop() {
    distance_F = Ultrasonic_read();
    Serial.print("D F="); Serial.println(distance_F);

    int right_val = digitalRead(R_S);
    int left_val = digitalRead(L_S);

    if ((right_val == 0) && (left_val == 0)) {
        if (distance_F <= Set) {
            Check_side();
        } else {

            if (lastAction == 'R') {
                turnRight();
            } else if (lastAction == 'L') {
                turnLeft();
            } else {
                forward();
            }
        }
    }

    else if ((right_val == 0) && (left_val == 1)) {
        turnRight();
        lastAction = 'R';
    }

    else if ((right_val == 1) && (left_val == 0)) {
        turnLeft();
        lastAction = 'L';
    }

    else if ((right_val == 1) && (left_val == 1)) {
        forward();
        lastAction = 'F';
    }

    delay(10);
}

```

```

// ===== Servo Control =====
void servoPulse(int pin, int angle) {
    int pwm = (angle * 11) + 500; // Convert angle to microseconds
    digitalWrite(pin, HIGH);
    delayMicroseconds(pwm);
    digitalWrite(pin, LOW);
    delay(50); // Refresh cycle
}

// ===== Ultrasonic Sensor =====
long Ultrasonic_read() {
    digitalWrite(trigger, LOW);
    delayMicroseconds(2);
    digitalWrite(trigger, HIGH);
    delayMicroseconds(10);
    long time = pulseIn(echo, HIGH);
    return time / 29 / 2; // Convert to cm
}

// ===== Obstacle Avoidance =====
void compareDistance() {
    if (distance_L > distance_R) {
        turnLeft(); delay(500);
        forward(); delay(600);
        turnRight(); delay(600);

        forward(); delay(600);
        turnRight();
        while(digitalRead(R_S) == 0 && digitalRead(L_S) == 0) {
            delay(10); // انتظر حتى يجد الخط
        }
    } else {
        turnRight(); delay(500);
        forward(); delay(600);
        turnLeft(); delay(600);
        forward(); delay(600);
        turnLeft();
        while(digitalRead(R_S) == 0 && digitalRead(L_S) == 0) {
            delay(10); // انتظر حتى يجد الخط
        }
    }
}

void Check_side() {
    Stop(); delay(100);
}

```

```

    for (int angle = 70; angle <= 140; angle += 5) servoPulse(servo,
angle);
    delay(300);
    distance_R = Ultrasonic_read();
    Serial.print("D R="); Serial.println(distance_L);

    for (int angle = 140; angle >= 0; angle -= 5) servoPulse(servo,
angle);
    delay(500);
    distance_L = Ultrasonic_read();
    Serial.print("D L="); Serial.println(distance_R);

    for (int angle = 0; angle <= 70; angle += 5) servoPulse(servo,
angle);
    delay(300);

    compareDistance();
}

// ===== Motor Control =====
void forward() {
    digitalWrite(in1, LOW);
    digitalWrite(in2, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
}

void backward() {
    digitalWrite(in1, HIGH);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
}

void turnRight() {
    digitalWrite(in1, LOW);
    digitalWrite(in2, HIGH);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
}

```

```

void turnLeft() {
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
}

void Stop() {
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
}

```

Part 3: Figures

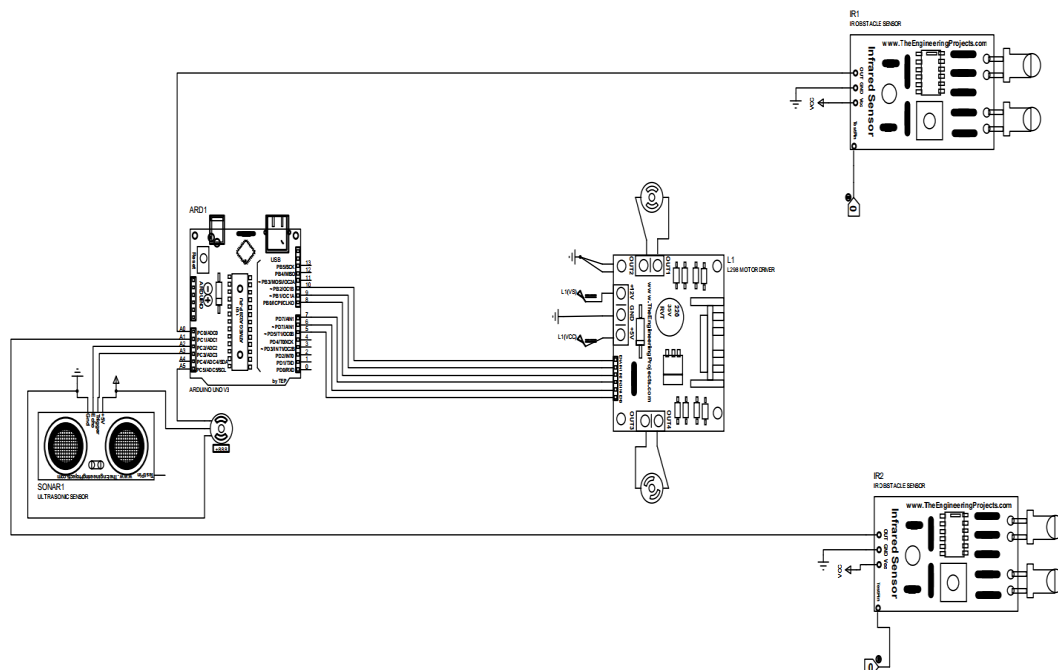


Figure Y: Proteus simulation schematic illustrating the robot hardware architecture and sensor interfacing.

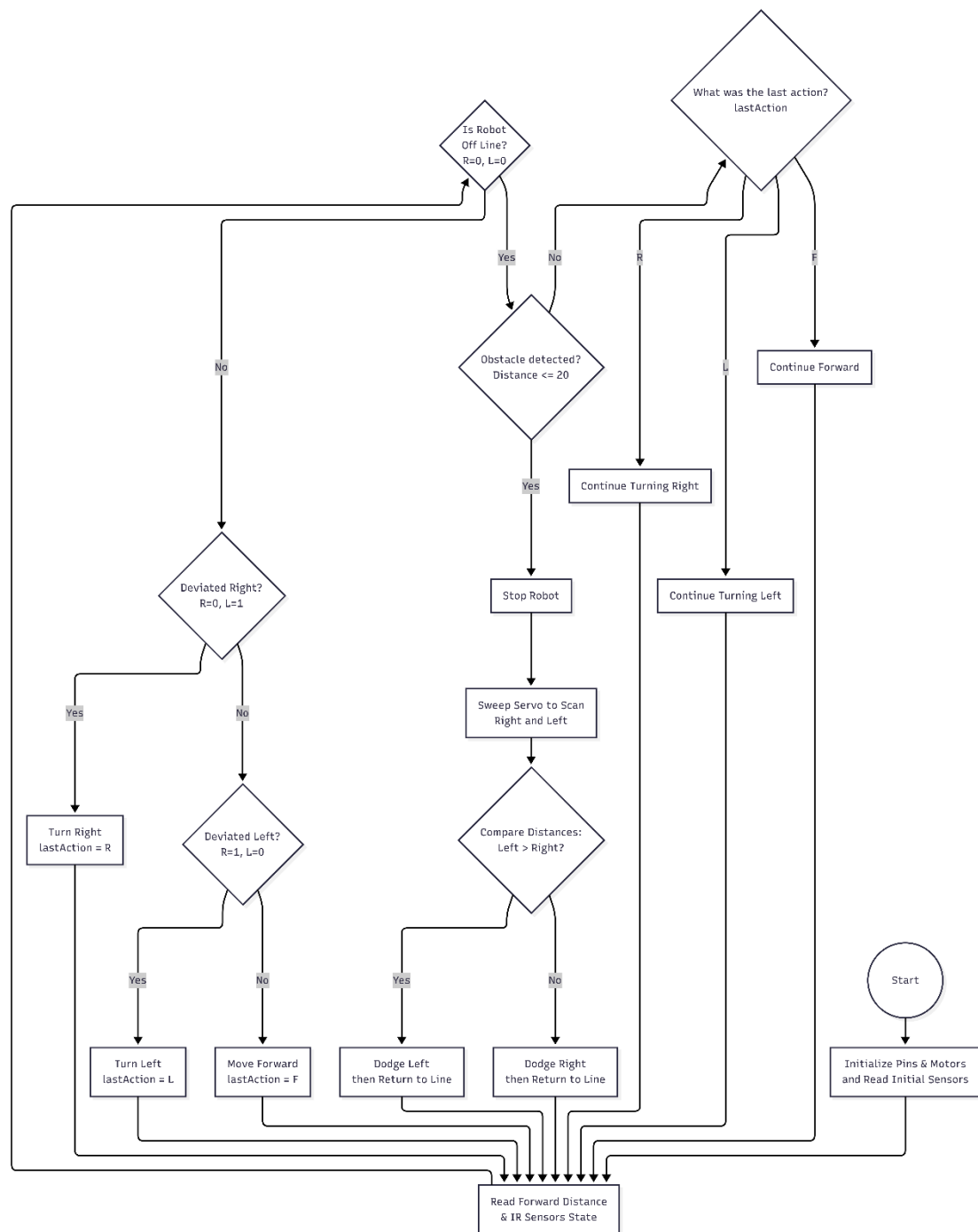


Figure 1: Main decision-making process for the autonomous navigation syste

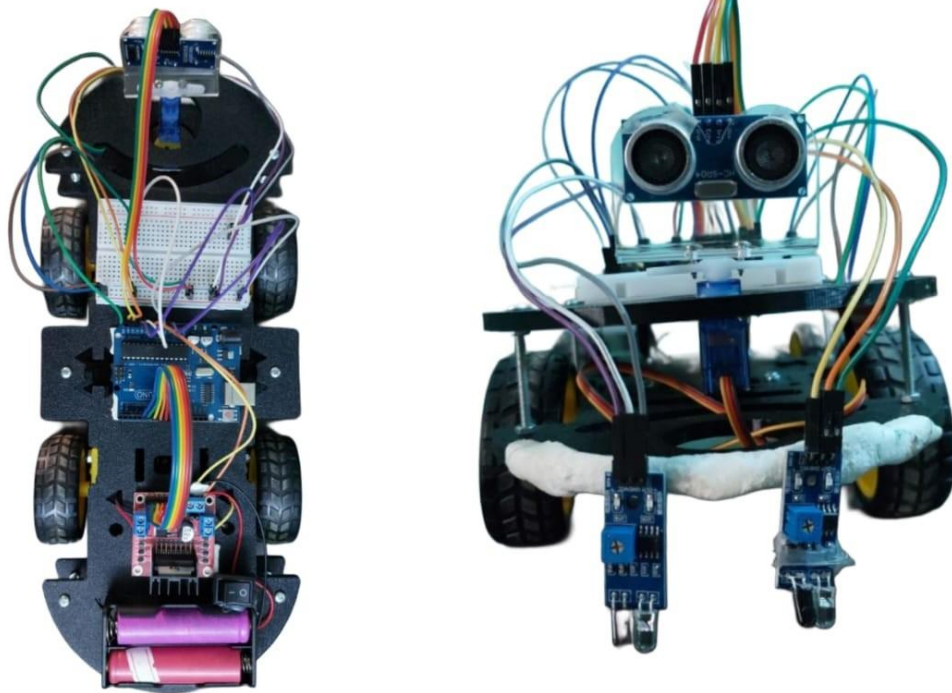
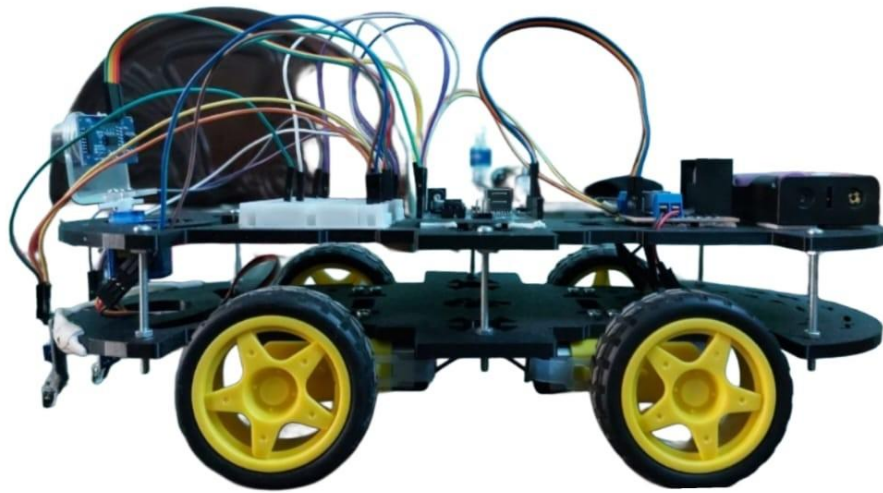


Figure X: Final hardware implementation of the autonomous mobile robot showing the Arduino Uno, L298N motor driver, power system, breadboard connections, and ultrasonic sensing module.